WEB TECHNOLOGY

7 Practical Programs — Complete Code + Procedure + Commands + How to Run

No.	Practical	How it runs
1	Create Form Validation Using JavaScript	Browser
2	AngularJS Shopping List Application	Browser
3	Simple Calculator Application Using React JS	React
4	NodeJS Server Without Express	NodeJS
5	Counter Using ReactJS	React
6	Simple Login Form Using React JS	React
7	Chat Module Using HTML, CSS and JavaScript	Browser

Exam-use format: For each practical, first read the Aim, Requirements, Commands, Procedure, then create the files, paste the code, run it, and verify the Result. Commands are included separately so you can directly type them in Command Prompt/PowerShell.

Practical 1: Create Form Validation Using JavaScript

Aim: To create Form validation using JavaScript.

Requirements: Browser + Notepad/VS Code. No Node.js is required.

Commands / How to Run

• Windows: create a folder, then save the file as form_validation.html.

• Run by double-clicking form_validation.html, or right-click → Open with → Chrome/Edge.

• Optional VS Code: open the folder and use Live Server if installed.

Step-by-Step Procedure

1. Create a folder named Practical1_FormValidation.

2. Create form_validation.html inside it.

3. Paste the complete code below and save.

4. Open the HTML file in a browser.

5. Test empty fields, invalid email, invalid phone, and valid details.

6. Click Reset to clear the form.

Program Code

form_validation.html

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <title>Registration Form</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            background: #f4f4f4;
        }

        h1 {
            text-align: center;
        }

        form {
            width: 600px;
            margin: 20px auto;
            background: white;
            padding: 20px;
            border-radius: 8px;
        }

        input, select, textarea {
            padding: 8px;
            box-sizing: border-box;
        }

        input[type="text"],
        input[type="password"],
        select,
        textarea {
            width: 100%;
        }

        input[type="submit"],
        input[type="reset"] {
            width: auto;
            cursor: pointer;
            margin-right: 8px;
        }
    </style>

    <script>
        function VALIDATEDETAIL() {
            var form = document.forms["RegForm"];
            var name = form["Name"];
            var email = form["EMail"];
            var phone = form["Telephone"];
            var course = form["Subject"];
            var password = form["Password"];
            var address = form["Address"];

            if (name.value.trim() === "") {
                alert("Please enter your name.");
                name.focus();
                return false;
```

```
            }

            if (address.value.trim() === "") {
                alert("Please enter your address.");
                address.focus();
                return false;
            }

            if (email.value.trim() === "") {
                alert("Please enter a valid e-mail address.");
                email.focus();
                return false;
            }

            var emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;

            if (!emailPattern.test(email.value.trim())) {
                alert("Please enter a valid e-mail address.");
                email.focus();
                return false;
            }

            if (!/^\d{10}$/.test(phone.value.trim())) {
                alert("Please enter a valid 10-digit telephone number.");
                phone.focus();
                return false;
            }

            if (password.value === "") {
                alert("Please enter your password.");
                password.focus();
                return false;
            }

            if (course.value === "") {
                alert("Please select your course.");
                course.focus();
                return false;
            }

            alert("Form submitted successfully!");
            return true;
        }
    </script>
</head>

<body>
    <h1>REGISTRATION FORM</h1>

    <form name="RegForm" onsubmit="return VALIDATEDETAIL()">
        <p>Name: <input type="text" name="Name"></p>
        <p>Address: <input type="text" name="Address"></p>
        <p>E-mail Address: <input type="text" name="EMail"></p>
        <p>Password: <input type="password" name="Password"></p>
        <p>Telephone: <input type="text" name="Telephone"></p>

        <p>
            Select Your Course:
            <select name="Subject">
                <option value="">-- Select Course --</option>
                <option value="BTECH">BTECH</option>
                <option value="BBA">BBA</option>
                <option value="BCA">BCA</option>
                <option value="BCOM">B.COM</option>
            </select>
        </p>

        <p>Comments: <textarea name="Comment" rows="4"></textarea></p>

        <p>
            <input type="submit" value="Send">
            <input type="reset" value="Reset">
        </p>
    </form>
</body>
</html>
```

Expected Output / Result

Registration form appears; invalid details produce alerts and valid details show a success message.

Practical 2: AngularJS Shopping List Application

Aim: To develop an AngularJS application that displays shopping items and allows users to add and remove items.

Requirements: Browser + text editor. Internet connection is required for the AngularJS CDN used in the supplied program.

Commands / How to Run

• No terminal command is required.

• Create index.html and open it directly in Chrome/Edge.

• Optional: in VS Code, use Live Server if available.

Step-by-Step Procedure

1. Create a folder named Practical2_AngularShopping.

2. Create index.html.

3. Paste the code below and save.

4. Open index.html in the browser.

5. Enter an item and click Add Item.

6. Click Remove to delete an item.

7. If the list is empty, No items in the list is displayed.

Program Code

index.html

```
<!DOCTYPE html>
<html lang="en" ng-app="shoppingApp">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Shopping List App</title>
    <script src="https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js"></script>

    <style>
        body {
            font-family: Arial, sans-serif;
            padding: 20px;
            background: #f9f9f9;
        }

        h2 { text-align: center; }

        .shopping-list {
            max-width: 500px;
            margin: 0 auto;
            background: #fff;
            padding: 20px;
            border-radius: 8px;
            box-shadow: 0 4px 8px rgba(0,0,0,0.1);
        }

        .shopping-list input {
            width: 70%;
            padding: 8px;
        }

        button {
            padding: 8px 16px;
            border: none;
            border-radius: 4px;
            background: #4CAF50;
            color: white;
            cursor: pointer;
        }

        .item {
            display: flex;
            justify-content: space-between;
            margin: 10px 0;
        }

        .item button { background: #f44336; }
    </style>
</head>

<body ng-controller="shoppingCtrl">
    <h2>Shopping List</h2>
```

```
    <div class="shopping-list">
        <input type="text" ng-model="newItem" placeholder="Enter item">
        <button ng-click="addItem()">Add Item</button>

        <div ng-if="shoppingList.length > 0">
            <h3>Items:</h3>
            <ul>
                <li ng-repeat="item in shoppingList track by $index">
                    <div class="item">
                        <span>{{ item }}</span>
                        <button ng-click="removeItem($index)">Remove</button>
                    </div>
                </li>
            </ul>
        </div>

        <div ng-if="shoppingList.length === 0">
            <p>No items in the list.</p>
        </div>
    </div>

    <script>
        var app = angular.module("shoppingApp", []);

        app.controller("shoppingCtrl", function($scope) {
            $scope.shoppingList = [];

            $scope.addItem = function() {
                if ($scope.newItem && $scope.newItem.trim() !== "") {
                    $scope.shoppingList.push($scope.newItem.trim());
                    $scope.newItem = "";
                }
            };

            $scope.removeItem = function(index) {
                $scope.shoppingList.splice(index, 1);
            };
        });
    </script>
</body>
</html>
```

Expected Output / Result

Shopping List appears; items can be added and removed.

Practical 3: Simple Calculator Application Using React JS

Aim: To create a simple calculator application using React JS.

Requirements: Node.js + npm + a React project.

Commands / How to Run

• Check Node.js: node -v

• Check npm: npm -v

• Create project: npx create-react-app calculator

• Enter project: cd calculator

• Run: npm start

• Stop: Ctrl + C

Step-by-Step Procedure

1. Install Node.js if it is not already installed.

2. Open Command Prompt/PowerShell.

3. Run node -v and npm -v to verify installation.

4. Run npx create-react-app calculator.

5. Run cd calculator.

6. Replace src/App.js and src/App.css with the code below.

7. Run npm start.

8. The React development server opens the local application in the browser.

9. Test +, -, *, /, C and =.

Program Code

src/App.js

```
import React, { useState } from "react";
import "./App.css";

function App() {
    const [currentInput, setCurrentInput] = useState("");
    const [previousInput, setPreviousInput] = useState("");
    const [operation, setOperation] = useState("");

    const handleNumberClick = (number) => {
        setCurrentInput(currentInput + number);
    };

    const handleOperationClick = (operator) => {
        if (currentInput === "") return;
        setPreviousInput(currentInput);
        setOperation(operator);
        setCurrentInput("");
    };

    const handleClear = () => {
        setCurrentInput("");
        setPreviousInput("");
        setOperation("");
    };

    const handleEvaluate = () => {
        if (previousInput === "" || operation === "" || currentInput === "") return;

        const prev = parseFloat(previousInput);
        const current = parseFloat(currentInput);
        let result;

        switch (operation) {
            case "+":
                result = prev + current;
                break;
            case "-":
                result = prev - current;
                break;
            case "*":
                result = prev * current;
                break;
            case "/":
```

```
                if (current === 0) {
                    alert("Cannot divide by zero.");
                    return;
                }
                result = prev / current;
                break;
            default:
                return;
        }

        setCurrentInput(result.toString());
        setPreviousInput("");
        setOperation("");
    };

    return (
        <div className="calculator">
            <div className="display">
                <div className="previous-input">
                    {previousInput} {operation}
                </div>
                <div className="current-input">
                    {currentInput || "0"}
                </div>
            </div>

            <div className="buttons">
                <button onClick={() => handleNumberClick("7")}>7</button>
                <button onClick={() => handleNumberClick("8")}>8</button>
                <button onClick={() => handleNumberClick("9")}>9</button>
                <button onClick={() => handleOperationClick("/")}>/</button>

                <button onClick={() => handleNumberClick("4")}>4</button>
                <button onClick={() => handleNumberClick("5")}>5</button>
                <button onClick={() => handleNumberClick("6")}>6</button>
                <button onClick={() => handleOperationClick("*")}>*</button>

                <button onClick={() => handleNumberClick("1")}>1</button>
                <button onClick={() => handleNumberClick("2")}>2</button>
                <button onClick={() => handleNumberClick("3")}>3</button>
                <button onClick={() => handleOperationClick("-")}>-</button>

                <button onClick={() => handleNumberClick("0")}>0</button>
                <button onClick={handleClear}>C</button>
                <button onClick={handleEvaluate}>=</button>
                <button onClick={() => handleOperationClick("+")}>+</button>
            </div>
        </div>
    );
}

export default App;
```

src/App.css

```
.calculator {
    width: 260px;
    margin: 100px auto;
    background-color: #f4f4f4;
    padding: 20px;
    border-radius: 10px;
    box-shadow: 0 4px 8px rgba(0,0,0,0.2);
}

.display {
    text-align: right;
    margin-bottom: 20px;
}

.previous-input {
    font-size: 16px;
    color: #888;
    min-height: 20px;
}

.current-input {
    font-size: 32px;
    font-weight: bold;
    min-height: 38px;
}

.buttons {
    display: grid;
    grid-template-columns: repeat(4, 1fr);
    gap: 5px;
}

.buttons button {
    width: 50px;
    height: 50px;
    font-size: 18px;
    border: 1px solid #ccc;
    background: #fff;
    cursor: pointer;
    border-radius: 5px;
}
```

Expected Output / Result

Calculator interface appears and performs basic arithmetic.

Practical 4: NodeJS Server Without Express

Aim: To create a NodeJS server that serves static HTML and CSS files without using Express.

Requirements: Node.js + text editor.

Commands / How to Run

• Check Node.js: node -v

• Check npm: npm -v (not required by this program, but useful to verify Node installation)

• Run server: node server.js

• Open browser: http://127.0.0.1:3000

• Stop server: Ctrl + C

Step-by-Step Procedure

1. Install Node.js.

2. Create a folder named Practical4_NodeServer.

3. Create server.js and index.html in the same folder.

4. Paste the code below into the respective files.

5. Open Command Prompt/PowerShell in that folder.

6. Run node server.js.

7. The terminal should show Server is running at http://127.0.0.1:3000.

8. Open http://127.0.0.1:3000 in the browser.

9. Press Ctrl + C to stop the server.

Program Code

server.js

```
const http = require("http");
const url = require("url");
const fs = require("fs");
const path = require("path");

const mimeTypes = {
    ".html": "text/html",
    ".jpg": "image/jpeg",
    ".jpeg": "image/jpeg",
    ".css": "text/css",
    ".js": "text/javascript",
    ".png": "image/png",
    ".ico": "image/x-icon"
};

function getContentType(filePath) {
    const extension = path.extname(filePath).toLowerCase();
    return mimeTypes[extension] || "application/octet-stream";
}

const server = http.createServer((request, response) => {
    const parsedUrl = url.parse(request.url, true);
    let pathname = parsedUrl.pathname;

    if (pathname === "/") {
        pathname = "/index.html";
    }

    const filePath = path.join(__dirname, pathname);

    if (!filePath.startsWith(__dirname)) {
        response.writeHead(403, {"Content-Type": "text/plain"});
        response.end("403 - Forbidden");
        return;
    }

    const contentType = getContentType(filePath);

    fs.readFile(filePath, (error, data) => {
        if (error) {
            response.writeHead(404, {"Content-Type": "text/plain"});
            response.end("404 - File Not Found");
            return;
        }

        response.writeHead(200, {"Content-Type": contentType});
```

```
        response.end(data);
    });
});

const PORT = 3000;
const HOST = "127.0.0.1";

server.listen(PORT, HOST, () => {
    console.log(`Server is running at http://${HOST}:${PORT}`);
});
```

index.html

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <title>NodeJS Static Server</title>
</head>
<body>
    <h1>Hello from NodeJS Server</h1>
    <p>This HTML file is served without Express.</p>
</body>
</html>
```

Expected Output / Result

Terminal shows the server address and the browser displays the HTML page.

Practical 5: Counter Using ReactJS

Aim: To create a counter using ReactJS.

Requirements: Node.js + npm + React project.

Commands / How to Run

• Check Node: node -v

• Check npm: npm -v

• Create project: npx create-react-app counter

• Enter project: cd counter

• Run: npm start

• Stop: Ctrl + C

Step-by-Step Procedure

1. Open Command Prompt/PowerShell.

2. Verify Node.js and npm using node -v and npm -v.

3. Run npx create-react-app counter.

4. Run cd counter.

5. Replace src/App.js and src/App.css.

6. Run npm start.

7. Use Increment, Decrement and Reset.

Program Code

src/App.js

```
import React, { useState } from "react";
import "./App.css";

function App() {
    const [count, setCount] = useState(0);

    return (
        <div className="counter">
            <h1>Counter</h1>
            <h2>{count}</h2>

            <div className="buttons">
                <button onClick={() => setCount(count + 1)}>Increment</button>
                <button onClick={() => setCount(count - 1)}>Decrement</button>
                <button onClick={() => setCount(0)}>Reset</button>
            </div>
        </div>
    );
}

export default App;
```

src/App.css

```
.counter {
    text-align: center;
    margin-top: 100px;
    font-family: Arial, sans-serif;
}

.counter h2 {
    font-size: 50px;
    margin: 20px;
}

.buttons button {
    margin: 5px;
    padding: 10px 18px;
    font-size: 16px;
    cursor: pointer;
}
```

Expected Output / Result

Counter starts at 0 and changes with the three buttons.

Practical 6: Simple Login Form Using React JS

Aim: To create a simple login form using React JS.

Requirements: Node.js + npm + React project.

Commands / How to Run

• Check Node: node -v

• Check npm: npm -v

• Create project: npx create-react-app login-form

• Enter project: cd login-form

• Run: npm start

• Stop: Ctrl + C

Step-by-Step Procedure

1. Create the React project using the commands above.

2. Open the project folder.

3. Replace src/App.js and src/App.css with the code below.

4. Run npm start.

5. Test username user1, password pass1.

6. Also test user2/pass2.

7. Enter wrong credentials to check validation.

Program Code

src/App.js

```
import React, { useState } from "react";
import "./App.css";

function App() {
    const [errorMessages, setErrorMessages] = useState({});
    const [isSubmitted, setIsSubmitted] = useState(false);

    const database = [
        { username: "user1", password: "pass1" },
        { username: "user2", password: "pass2" }
    ];

    const errors = {
        uname: "Invalid username",
        pass: "Invalid password"
    };

    const handleSubmit = (event) => {
        event.preventDefault();

        const form = event.target;
        const uname = form.elements.uname.value;
        const pass = form.elements.pass.value;

        const userData = database.find(
            (user) => user.username === uname
        );

        if (!userData) {
            setErrorMessages({name: "uname", message: errors.uname});
            return;
        }

        if (userData.password !== pass) {
            setErrorMessages({name: "pass", message: errors.pass});
            return;
        }

        setErrorMessages({});
        setIsSubmitted(true);
    };

    const renderErrorMessage = (name) =>
        name === errorMessages.name && (
            <div className="error">{errorMessages.message}</div>
        );

    const renderForm = (
```

```
        <div className="form">
            <form onSubmit={handleSubmit}>
                <div className="input-container">
                    <label>Username</label>
                    <input type="text" name="uname" required />
                    {renderErrorMessage("uname")}
                </div>

                <div className="input-container">
                    <label>Password</label>
                    <input type="password" name="pass" required />
                    {renderErrorMessage("pass")}
                </div>

                <div className="button-container">
                    <input type="submit" value="Login" />
                </div>
            </form>
        </div>
    );

    return (
        <div className="app">
            <div className="login-form">
                <div className="title">Sign In</div>
                {isSubmitted ? (
                    <div>User is successfully logged in</div>
                ) : renderForm}
            </div>
        </div>
    );
}

export default App;
```

src/App.css

```
.app {
    font-family: Arial, sans-serif;
    display: flex;
    align-items: center;
    justify-content: center;
    min-height: 100vh;
    background: #f8f9fd;
}

.login-form {
    background: white;
    padding: 2rem;
    min-width: 280px;
    box-shadow: 0 4px 8px rgba(0,0,0,0.2);
}

.title {
    font-size: 25px;
    margin-bottom: 20px;
}

.input-container {
    display: flex;
    flex-direction: column;
    gap: 8px;
    margin: 10px 0;
}

input[type="text"],
input[type="password"] {
    height: 30px;
    padding: 4px 8px;
}

input[type="submit"] {
    margin-top: 10px;
    cursor: pointer;
    font-size: 15px;
    background: #01d28e;
    border: 1px solid #01d28e;
    color: white;
    padding: 10px 20px;
}

.button-container {
    display: flex;
    justify-content: center;
}

.error {
    color: red;
    font-size: 12px;
}
```

Expected Output / Result

Correct credentials show successful login; incorrect credentials show invalid username/password.

Practical 7: Chat Module Using HTML, CSS and JavaScript

Aim: To build a chat module using HTML, CSS and JavaScript.

Requirements: Browser + text editor. No Node.js is required for the basic version.

Commands / How to Run

• No terminal command is required.

• Create the three files in one folder.

• Open index.html directly in Chrome/Edge.

• Optional VS Code: use Live Server if installed.

Step-by-Step Procedure

1. Create a folder named Practical7_Chat.

2. Create index.html, style.css, and script.js.

3. Copy the code into the correct files.

4. Keep all three files in the same folder.

5. Open index.html in a browser.

6. Type a message.

7. Click Send or press Enter.

8. The message is added to the chat.

Program Code

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Simple Chat Module</title>
    <link rel="stylesheet" href="style.css">
</head>

<body>
    <div class="chat-box">
        <h2>Simple Chat</h2>

        <div id="chatMessages">
            <div class="container">
                <p>Hello. How are you today?</p>
                <span class="time-right">11:00</span>
            </div>

            <div class="container darker">
                <p>Hey! I'm fine. Thanks for asking!</p>
                <span class="time-left">11:01</span>
            </div>

            <div class="container">
                <p>Sweet! What do you want to do today?</p>
                <span class="time-right">11:02</span>
            </div>

            <div class="container darker">
                <p>Maybe learn more coding!</p>
                <span class="time-left">11:05</span>
            </div>
        </div>

        <div class="input-area">
            <input id="messageInput" type="text" placeholder="Type a message">
            <button id="sendButton">Send</button>
        </div>
    </div>

    <script src="script.js"></script>
</body>
</html>
```

style.css

```
body {
    font-family: Arial, sans-serif;
    background: #f5f5f5;
```

```
}

.chat-box {
    width: 600px;
    max-width: 90%;
    margin: 30px auto;
    background: white;
    padding: 20px;
    border-radius: 8px;
}

.container {
    border: 2px solid #dedede;
    background: #f1f1f1;
    border-radius: 5px;
    padding: 10px;
    margin: 10px 0;
}

.darker {
    border-color: #ccc;
    background: #ddd;
}

.container::after {
    content: "";
    clear: both;
    display: table;
}

.time-right {
    float: right;
    color: #aaa;
}

.time-left {
    float: left;
    color: #999;
}

.input-area {
    display: flex;
    gap: 10px;
    margin-top: 15px;
}

#messageInput {
    flex: 1;
    padding: 10px;
}

#sendButton {
    padding: 10px 18px;
    cursor: pointer;
}
```

script.js

```
const messageInput = document.getElementById("messageInput");
const sendButton = document.getElementById("sendButton");
const chatMessages = document.getElementById("chatMessages");

function sendMessage() {
    const message = messageInput.value.trim();

    if (message === "") {
        return;
    }

    const messageContainer = document.createElement("div");
    messageContainer.className = "container darker";

    const messageText = document.createElement("p");
    messageText.textContent = message;

    const time = document.createElement("span");
    time.className = "time-left";
    time.textContent = new Date().toLocaleTimeString([], {
        hour: "2-digit",
        minute: "2-digit"
    });

    messageContainer.appendChild(messageText);
    messageContainer.appendChild(time);
    chatMessages.appendChild(messageContainer);

    messageInput.value = "";
    messageInput.focus();
}

sendButton.addEventListener("click", sendMessage);

messageInput.addEventListener("keydown", function(event) {
    if (event.key === "Enter") {
        sendMessage();
    }
```

```
});
```

Expected Output / Result

Chat interface appears and new messages can be added.